

04 · Métricas

Quant-Backtester · material de estudio

Ya tienes una curva de equity por estrategia; ahora hay que resumirla en números comparables. En este capítulo implementas las métricas de la tabla del README a partir de una serie de rendimientos diarios: volatilidad anualizada, CAGR, Sharpe, Sortino, la serie de drawdown y el peor drawdown con sus fechas, Calmar, hit ratio y turnover anual. Ninguna es difícil de programar. Lo difícil es fijar las convenciones (ddof, cómo se cuentan los años, cómo se pasa el tipo libre de riesgo a diario, qué cuenta como pico) y entender qué supuestos esconde cada número.

El capítulo cubre los TODO 4.1 a 4.9 de `src/metrics.py`. Los tests están en `tests/test_metrics.py`, cuyos valores esperados se calculan a mano o con el módulo `statistics` de Python (sin pandas), y en `tests/test_quantstats_check.py`, que compara tus resultados con la librería `quantstats` con una tolerancia de `1e-10`.

1. Intuición

Qué pregunta responde cada número

Métrica	Pregunta que responde	Unidad
CAGR	¿A qué ritmo anual constante habría crecido el capital?	% anual
Volatilidad	¿Cuánto oscilan los rendimientos?	% anual
Sharpe	¿Cuánto exceso de rentabilidad hay por unidad de volatilidad?	anual, sin unidades
Sortino	Lo mismo, pero penalizando solo las caídas	anual, sin unidades
Max drawdown	¿Cuál fue la peor caída desde un máximo, y cuándo?	% (negativo)
Calmar	¿Cuánto crecimiento anual hay por unidad de peor caída?	sin unidades
Hit ratio	De los días con resultado, ¿qué fracción fueron ganadores?	%
Turnover anual	¿Cuántas veces al año se mueve el capital?	veces (x)

Las de rentabilidad y riesgo se usan en todas partes; el drawdown es la que mejor describe lo que se siente al aguantar una estrategia; hit ratio y turnover describen cómo opera, y el turnover es el puente con los costes.

Anualizar: la regla de la raíz cuadrada

Las métricas se calculan con datos diarios pero se publican en términos anuales. Si los rendimientos diarios son independientes y tienen la misma distribución (i.i.d.), la suma de 252 de ellos tiene una media 252 veces mayor y una varianza 252 veces mayor. La desviación típica, raíz de la varianza, crece con $\sqrt{252} \approx 15,87$: una volatilidad diaria del 1 % equivale a un 15,9 % anual. Por eso el Sharpe anual es el diario por $\sqrt{252}$ (media $\times 252$ entre $\sigma \times \sqrt{252}$). El 252 es la convención de sesiones por año (`config.TRADING_DAYS`), y todas las funciones lo reciben como parámetro `periods`.

Los dos supuestos fallan en la práctica, y conviene saber hacia dónde: con autocorrelación positiva (precios «suavizados», activos ilíquidos, estrategias de tendencia), $\sqrt{252}$ subestima la volatilidad anual; si la volatilidad cambia con el tiempo (en las crisis se dispara), la cifra anual es una media de regímenes muy distintos.

Media aritmética frente a crecimiento compuesto

Ganar un 50 % y luego perder un 50 % da una media aritmética del 0 %, pero te deja con el 75 % del capital. El Sharpe usa la media aritmética; el CAGR mide el crecimiento compuesto real. La diferencia aproximada entre los dos es $\sigma^2/2$ (el *volatility drag*), así que dos estrategias con la misma media pueden tener CAGR muy distintos si sus volatilidades son distintas.

Lo que ya está hecho

Todas las métricas llevan el decorador `@metric`. Antes de llamar a tu código limpia el primer argumento con `_as_series`: acepta una Series, un DataFrame de una sola columna (con más columnas lanza `TypeError` y te remite a `summary_table`) o directamente un `BacktestResult`, del que toma `.returns`; con `@metric(source="turnover")`, como en `annual_turnover`, toma `.turnover`. Después elimina los NaN y convierte a float. Así que dentro de cada TODO tienes una Series limpia de floats, y `n` es el número de observaciones válidas, no el de filas originales (`test_sharpe_ratio_ignores_nan` lo comprueba). Los demás argumentos (`rf`, `periods`) pasan tal cual.

`DrawdownInfo` es lo que devuelve `max_drawdown`: una dataclass inmutable con `depth`, `peak`, `trough` y `recovery`, más las propiedades `recovered` y `duration` (de pico a recuperación) y un `__str__` legible.

`summary_table` construye la tabla del README: una fila por estrategia y las columnas de `SUMMARY_COLUMNS`. Si una métrica sigue siendo un TODO, su columna sale NaN y se emite un aviso con los pendientes, sin romper el notebook; con Series en lugar de `BacktestResult` no hay turnover y esa columna queda NaN. `format_summary` y `to_markdown_table` la dejan lista para leer o pegar en el README. El diccionario `METRICS` es el registro que usa el walk-forward: siempre maximiza la métrica elegida (por defecto, el Sharpe).

`compare_with_quantstats` calcula tu Sharpe, Sortino, volatilidad y max drawdown y los de `quantstats` sobre la misma serie, con una columna de diferencias que debería rondar `1e-12` (los tests exigen menos de `1e-10`). Una diferencia de ese tamaño es redondeo; una mayor es una convención distinta. El CAGR no se compara. El docstring lo justifica diciendo que `quantstats` cuenta años de calendario: era así en versiones antiguas de la librería (años = días naturales entre la primera y la última fecha / 365), mientras que las recientes cuentan observaciones, como aquí. Como depende de la versión instalada, se deja fuera.

2. Formalización

Notación: `r_1, ..., r_n` son los rendimientos diarios ya limpios, `P = periods` (252 por defecto) y `rf` el tipo libre de riesgo **anual** (`rf`).

Volatilidad anualizada

$$s = \sqrt{\frac{1}{n-1} \sum_{t=1}^n (r_t - \bar{r})^2}$$
$$\sigma_{\text{anual}} = s \cdot \sqrt{P}$$

Se divide entre `n - 1` (ddof = 1, desviación muestral) porque la media se estima con la misma muestra y eso hace que las desviaciones respecto a ella sean, en promedio, algo

menores que respecto a la media verdadera: dividir entre n subestimaría la varianza. Con $[0,01; -0,01]$ la muestral es $\sqrt{2} \times 0,01 \approx 0,01414$ y la poblacional, 0,01; es lo que comprueba `test_annualized_volatility_uses_sample_std`.

Ejemplo con la serie de `tests/test_metrics.py`, $R = [0,01; -0,02; 0,03; 0; -0,01; 0,02]$, cuya media es $0,03 / 6 = 0,005$:

t	r	r - 0,005	(r - 0,005) ²	min(r, 0) ²
1	0,01	0,005	0,000025	0
2	-0,02	-0,025	0,000625	0,0004
3	0,03	0,025	0,000625	0
4	0,00	-0,005	0,000025	0
5	-0,01	-0,015	0,000225	0,0001
6	0,02	0,015	0,000225	0
Σ	0,03	0	0,00175	0,0005

De ahí: $s = \sqrt{(0,00175 / 5)} = \sqrt{0,00035} \approx 0,018708$ y la σ anual $\approx 0,018708 \times 15,8745 \approx 0,29698$, un 29,7 %. Con $\text{ddof} = 0$ saldría $\sqrt{(0,00175 / 6)} \approx 0,017078$, un 27,1 %. La última columna la usarás en el Sortino.

Por qué $\sqrt{P}$ y cuándo falla

Si los rendimientos son i.i.d. con varianza σ^2 , las covarianzas son cero y:

$$\text{Var}\left(\sum_{t=1}^P r_t\right) = P\sigma^2$$

Si los rendimientos de días consecutivos tienen una autocorrelación ρ (la que lleva el subíndice 1 en la fórmula) y se ignoran retardos mayores, aparecen los términos cruzados:

$$\text{Var}\left(\sum_{t=1}^P r_t\right) \approx P\sigma^2(1 + 2\rho_1)$$

Con $\rho = 0,1$ el factor es 1,2: la volatilidad anual real es $\sqrt{1,2} \approx 1,095$ veces la calculada con $\sqrt{252}$, que la subestima en torno a un 9 %. Con autocorrelación negativa (reversión a la media) ocurre lo contrario.

CAGR

$$\text{CAGR} = \left(\prod_{t=1}^n (1 + r_t)\right)^{P/n} - 1$$

El producto es el crecimiento total del capital, y n/P es el número de años contado en sesiones: con 252 rendimientos, un año. Los casos de `test_cagr_known_values`:

Rendimientos (n)	Crecimiento total	Años = n/252	CAGR
252	1,10	1	$1,10 - 1 = 10 \%$
504	1,21	2	$1,21^{(1/2)} - 1 = 10 \%$
126	1,05	0,5	$1,05^2 - 1 = 10,25 \%$

Contar años con el calendario es otra convención, no un error, pero da otro número. Las 252 sesiones que empiezan el 01-01-2010 acaban el 20-12-2010, 353 días naturales después: con años = $353/365 \approx 0,967$, un crecimiento del 10 % se convierte en un CAGR del 10,36 %. El proyecto cuenta sesiones, coherente con $\sqrt{252}$ en el resto de métricas.

Anualizar periodos cortos da cifras sin sentido aunque la fórmula sea correcta. La serie de seis días de `test_calmar_ratio_known_value` crece un 15,54 % en total, y elevar $1,1554$ a $252/6 = 42$ da un CAGR de alrededor del 43 035 %.

La relación aproximada con la media aritmética anual μ y la volatilidad anual σ :

$$\text{CAGR} \approx \mu - \frac{\sigma^2}{2}$$

Con $\mu = 10\%$ y $\sigma = 20\%$, el CAGR ronda el 8 %.

Tipo libre de riesgo y exceso de rentabilidad

El rf de `config.RISK_FREE` es anual. El equivalente diario es el que, compuesto P veces, da exactamente rf:

$$\text{rf}_d = (1 + \text{rf})^{1/P} - 1$$

$$x_t = r_t - \text{rf}_d$$

Dividir rf entre 252 es solo una aproximación lineal: compuesto 252 veces, $5\%/252$ da $(1 + 0,05/252)^{252} - 1 \approx 5,127\%$, no un 5 %. La diferencia diaria es pequeña (0,0001984 frente a 0,0001936, unos $4.8\text{e-}6$), y en el Sharpe de una estrategia con un 15 % de volatilidad equivale a unas 0,008 unidades ($4.8\text{e-}6 \times 252 / 0,15$). Económicamente es irrelevante, pero `test_sharpe_ratio_geometric_risk_free` (tolerancia relativa $1\text{e-}9$) y la comparación con quantstats ($1\text{e-}10$) lo detectan. Con la serie R y $\text{rf} = 5\%$, el Sharpe geométrico es 4,0783 y el aproximado, 4,0743.

Restar una constante no cambia la desviación típica, así que en el Sharpe `s_x = s`. En el Sortino sí importa: con $\text{rf} > 0$, un día con rendimiento 0 pasa a tener exceso negativo.

Ratio de Sharpe

$$\text{Sharpe} = \frac{\bar{x}}{s_x} \sqrt{P}$$

Con la serie R y $\text{rf} = 0$: $0,005 / 0,018708 \times 15,8745 \approx 4,2426$. Con $\text{ddof} = 0$ saldría 4,6476. Si `s_x = 0` (una serie constante), el cociente no tiene sentido y la función devuelve NaN.

Ratio de Sortino

$$\sigma_{\text{bajista}} = \sqrt{\frac{1}{n} \sum_{t=1}^n \min(x_t, 0)^2}$$

$$\text{Sortino} = \frac{\bar{x}}{\sigma_{\text{bajista}}} \sqrt{P}$$

La idea de Sortino y Price (1994) es que la volatilidad al alza no es riesgo: solo se penalizan los rendimientos por debajo de un objetivo, aquí 0 sobre el exceso. Fíjate en el denominador de la raíz: **n, el número total de días**, no el número de días negativos. Es la convención de quantstats (que sigue la nota «Sortino: A Sharper Ratio» de Red Rock Capital) y la del

docstring.

Con la serie R: $\sum \min(r, 0)^2 = 0,0005$, así que la σ bajista es $\sqrt{(0,0005 / 6)} \approx 0,0091287$ y el Sortino es $0,005 / 0,0091287 \times 15,8745 \approx 8,6948$. Si dividieras solo entre los 2 días negativos, la σ bajista sería $\sqrt{(0,0005 / 2)} \approx 0,0158114$ y el Sortino bajaría a 5,0200.

¿Por qué n total? El cuadrado de la σ bajista es un valor esperado sobre todos los días (un momento parcial inferior): los días sin pérdidas forman parte de la distribución y aportan riesgo cero. Dividir solo entre los negativos mide el tamaño medio de las pérdidas e ignora con qué frecuencia ocurren: una estrategia que pierde un 1 % un día de cada cien y otra que lo pierde la mitad de los días tendrían la misma σ bajista. Si no hay ningún exceso negativo, la σ bajista es 0 y la función devuelve NaN.

Drawdown

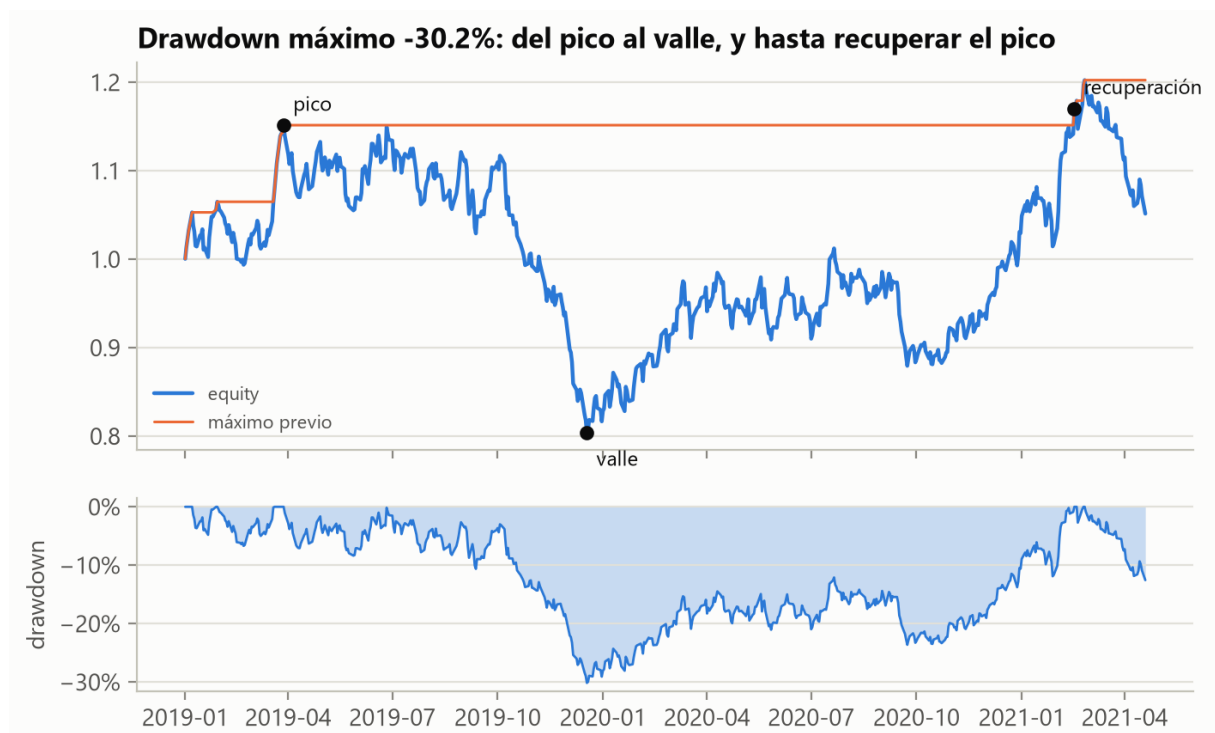
Partiendo de un capital de 1, la equity y su máximo previo son:

$$E_t = \prod_{s=1}^t (1 + r_s)$$
$$M_t = \max\left(1, \max_{s \leq t} E_s\right)$$
$$DD_t = \frac{E_t}{M_t} - 1$$

El 1 dentro del máximo es el **capital inicial como primer pico**: si pierdes desde el primer día, ya estás en drawdown. Ejemplo:

Día	r_t	E_t	M_t	DD_t
1	-4 %	0,960000	1,000000	-4,00 %
2	+2 %	0,979200	1,000000	-2,08 %
3	+3 %	1,008576	1,008576	0
4	-5 %	0,958147	1,008576	-5,00 %
5	+6 %	1,015636	1,015636	0

Sin el suelo de 1, el máximo del día 1 sería 0,96 (el propio día) y el día 2 marcaría un máximo nuevo: los dos primeros días aparecerían con drawdown 0 aunque estás por debajo del capital con el que empezaste. Aquí el peor drawdown no cambia, pero si la serie acabara en el día 2 pasarías de -4 % a 0.



Equity con su máximo acumulado (arriba) y drawdown (abajo). Se marcan el pico, el valle y la recuperación del peor drawdown.

Max drawdown y sus fechas

$$\text{MDD} = \min_t \text{DD}_t$$

Además de la profundidad interesan tres fechas: el **valle** (el día del mínimo; si se repite, el primero), el **pico** (el último día anterior o igual al valle con $\text{DD} = 0$) y la **recuperación** (el primer día posterior al valle con $\text{DD} = 0$). En la tabla: MDD = -5 %, valle el día 4, pico el día 3, recuperación el día 5.

Hay dos casos con **None**. Si antes del valle no hay ningún día con $\text{DD} = 0$, el pico fue el capital inicial, que no tiene fecha: con $[-4 \%, +2 \%, +5 \%]$ el valle es el día 1, el pico **None** y la recuperación el día 3 (equity 1,02816). Si después del valle nunca se vuelve a $\text{DD} = 0$, la recuperación es **None**: la estrategia sigue «dentro» del drawdown. Y si no hay ninguna caída ($\text{MDD} = 0$), las tres fechas son **None**.

Comparar DD_t con 0 de forma exacta es seguro aquí: cuando E_t es el máximo, $E_t / E_t - 1$ da exactamente 0.0 en coma flotante.

Calmar, hit ratio y turnover

$$\text{Calmar} = \frac{\text{CAGR}}{|\text{MDD}|}$$

Mide cuánto crecimiento anual compra cada punto de peor caída: con un CAGR del 8 % y un MDD del -20 %, Calmar = 0,4. Lo propuso Terry Young (1991) sobre los últimos 36 meses; aquí se usa toda la muestra. Ojo: el MDD solo puede empeorar al alargar el histórico, así que el Calmar de periodos largos tiende a ser menor. Sin drawdown, NaN.

$$\text{hit} = \frac{n_+}{n_+ + n_-}$$

Aquí `n_+` es el número de días con rendimiento positivo y `n_-`, el de días con rendimiento negativo; los días planos (exactamente 0) no cuentan. Con la serie R: $3 / (3 + 2) = 60 \%$; contando el día plano saldría $3/6 = 50 \%$. La razón para excluirlas: un día con rendimiento 0 suele ser un día fuera del mercado, que no es ni acierto ni fallo; una estrategia que está fuera el 70 % del tiempo parecería mucho peor de lo que es. Un hit ratio por sí solo no dice si se gana dinero: con un 40 % de aciertos se gana si las ganancias medias superan con holgura a las pérdidas medias. Si todos los días son planos, NaN.

$$TO_{\text{anual}} = \frac{\sum_{t=1}^n \tau_t}{n/P}$$

Aquí τ es el turnover diario de la cartera (`BacktestResult.turnover`, ya dividido entre el número de activos por `to_portfolio`), y n/P son los mismos años que en el CAGR. En `test_annual_turnover_known_value`, 504 días con un turnover total de 10 dan 5 al año. El coste anual aproximado es $\text{turnover anual} \times (\text{comisión} + \text{slippage}) / 10\,000$: con un turnover anual de 5 y $5 + 5$ bps, un 0,5 % al año.

Limitaciones del Sharpe

- Colas gruesas y asimetría. La desviación típica trata igual las subidas y las bajadas e ignora la forma de las colas. Una estrategia que vende opciones muy fuera de dinero cobra primas pequeñas y regulares (Sharpe alto) hasta el día del desplome. Mira también el max drawdown, el Sortino y la asimetría de la distribución.
- Autocorrelación. Con rendimientos autocorrelacionados positivamente, $\sqrt{252}$ subestima la volatilidad y el Sharpe sale inflado (sección anterior). Lo (2002) da la corrección; quantstats incluye un `smart_sharpe` que la penaliza.
- Error de estimación. Un Sharpe calculado con pocos años es una estimación muy ruidosa, como se ve a continuación.

Con rendimientos i.i.d., el error estándar del Sharpe por periodo es aproximadamente $\sqrt{(1 + SR^2/2)/T}$, con SR el Sharpe diario y T el número de observaciones. Como el Sharpe diario es pequeño, al anualizar queda una regla muy simple, con Y el número de años:

$$SE_{\text{anual}} \approx \frac{1}{\sqrt{Y}}$$

Con 10 años, $\pm 0,32$: un Sharpe de 0,5 y otro de 0,7 son indistinguibles. Y si eliges el mejor de muchos intentos (como en el grid search del capítulo 05), el máximo está sesgado al alza.

Los números del ejemplo son absurdos a propósito

Un Sharpe de 4,24 o una volatilidad del 29,7 % calculados con seis días no significan nada: la serie R solo sirve para comprobar la aritmética. Con datos reales, un Sharpe anual sostenido por encima de 1 ya es excepcional.

3. Del papel al código

Todas las funciones de esta sección llevan el decorador `@_metric` (o `@_metric(source="turnover")`), que se omite en las firmas. Usa siempre el parámetro `periods`, nunca un 252 escrito a mano.

TODO 4.1 · annualized_volatility

Qué hace. Desviación típica muestral de los rendimientos diarios, multiplicada por $\sqrt{\text{periods}}$.

Firma.

```
def annualized_volatility(returns: pd.Series, periods: int = config.TRADING_DAYS) -> float:
```

Convenciones que comprueban los tests.

- `test_annualized_volatility_known_value`: la desviación muestral de R (el test la calcula con el módulo `statistics`) por $\sqrt{252}$.
- `test_annualized_volatility_uses_sample_std`: `ddof = 1` (`[0.01, -0.01]` da $\sqrt{2} \times 0,01 \times \sqrt{252}$).
- Coincide con la «Volatilidad» de quantstats en `tests/test_quantstats_check.py`.

Pseudocódigo.

1. `s` = desviación típica muestral (`n - 1`) de los rendimientos.
2. Devuelve `s * raíz(periods)`.

Pistas.

Pista 1

Dispersión diaria, escalada por la raíz del número de días del año.

Pista 2

`Series.std()` y `np.sqrt`.

Pista 3

`Series.std` usa `ddof = 1` por defecto, pero `np.std` usa `ddof = 0`: si pasas por NumPy, indícalo.

TODO 4.2 · cagr

Qué hace. Tasa anual constante que reproduce el crecimiento total, con los años contados como `n / periods`.

Firma.

```
def cagr(returns: pd.Series, periods: int = config.TRADING_DAYS) -> float:
```

Convenciones que comprueban los tests.

- `test_cagr_known_values` (ids «1 año», «2 años», «medio año»): 10 %, 10 % y $1,05^2 - 1$.
- Los años salen del número de observaciones, no de las fechas del índice.

Pseudocódigo.

1. crecimiento = producto de $(1 + r)$ en todos los días.
2. años = $n / \text{periods}$.
3. Devuelve crecimiento elevado a $(1 / \text{años})$, menos 1.

Pistas.

Pista 1

Busca la tasa anual que, compuesta durante los años de la muestra, deja el mismo capital final.

Pista 2

`Series.prod()` y `len()`.

Pista 3

El exponente es $\text{periods} / n$, no $n / \text{periods}$: con medio año tiene que valer 2.

TODO 4.3 · sharpe_ratio

Qué hace. Media del exceso diario sobre su desviación típica muestral, anualizada con $\sqrt{\text{periods}}$, con r_f anual convertido a diario de forma geométrica.

Firma.

```
def sharpe_ratio(
    returns: pd.Series, rf: float = config.RISK_FREE, periods: int = config.TRADING_DAYS
) -> float:
```

Convenciones que comprueban los tests.

- `test_sharpe_ratio_known_value`: con $r_f = 0$, media de R entre su desviación muestral, por $\sqrt{252}$ (4,2426 en la sección 2).
- `test_sharpe_ratio_geometric_risk_free`: con $r_f = 5\%$, $r_{f_d} = 1,05^{(1/252)} - 1$, tolerancia relativa $1e-9$.
- `test_sharpe_ratio_constant_series_is_nan`: diez ceros dan NaN.
- `test_sharpe_ratio_ignores_nan`: un NaN inicial no cambia el resultado (lo resuelve el decorador).
- `test_compare_with_quantstats_matches` con $r_f = 0$ y $r_f = 3\%$.

Pseudocódigo.

1. $r_{f_d} = (1 + r_f)$ elevado a $(1 / \text{periods})$, menos 1.
2. exceso = rendimientos - r_{f_d} .
3. s = desviación típica muestral del exceso; si s es 0, devuelve NaN.
4. Devuelve $\text{media}(\text{exceso}) / s * \text{raíz}(\text{periods})$.

Pistas.

Pista 1

Exceso de rentabilidad por unidad de riesgo, primero en diario y luego anualizado.

Pista 2

`mean()`, `std()`, `np.sqrt` y `np.nan`.

Pista 3

Comprueba σ antes de dividir: 0/0 en NumPy da NaN con un aviso y x/0 da infinito, que no es lo que pide el docstring. Ojo también: una serie constante distinta de 0 (diez valores 0.001) da una σ de unos $2e-19$ por redondeo, no 0 exacto; el test solo usa ceros.

TODO 4.4 · sortino_ratio

Qué hace. Como el Sharpe, pero con la desviación bajista del exceso en el denominador, calculada sobre todos los días.

Firma.

```
def sortino_ratio(
    returns: pd.Series, rf: float = config.RISK_FREE, periods: int = config.TRADING_DAYS
) -> float:
```

Convenciones que comprueban los tests.

- `test_sortino_ratio_known_value`: σ bajista = $\sqrt{(\sum \min(r, 0)^2 / n)}$ con n = todos los días.
- `test_sortino_ratio_with_risk_free`: con $rf = 3\%$, el mínimo se toma sobre el exceso, no sobre el rendimiento.
- `test_sortino_ratio_without_losses_is_nan`: [0.01, 0.02, 0.0] da NaN.
- La comparación con quantstats, con $rf = 0$ y $rf = 3\%$.

Pseudocódigo.

1. exceso como en el Sharpe.
2. Para cada día, el mínimo entre el exceso y 0, al cuadrado.
3. $\sigma_{\text{bajista}} = \text{raíz}(\text{suma de esos cuadrados} / n)$, con n = todos los días.
4. Si σ_{bajista} es 0, devuelve NaN.
5. Devuelve $\text{media}(\text{exceso}) / \sigma_{\text{bajista}} * \text{raíz}(\text{periods})$.

Pistas.

Pista 1

El numerador es el mismo que en el Sharpe; solo cambia el denominador.

Pista 2

`clip(upper=0)` calcula $\min(x, 0)$ elemento a elemento.

Pista 3

$\sum \min(x, 0)^2 / n$ es una media sobre todos los días en la que los días positivos valen 0; no es una media sobre los negativos ni la desviación típica de los negativos.

TODO 4.5 · drawdown_series

Qué hace. Para cada día, la caída de la equity respecto a su máximo previo, contando el capital inicial como pico.

Firma.

```
def drawdown_series(returns: pd.Series) -> pd.Series:
```

Convenciones que comprueban los tests.

- `test_drawdown_series_known_values`: [0.10, -0.10, 0.05, 0.10] da [0, -10 %, -5,5 %, 0].
- `test_drawdown_series_initial_capital_is_a_peak`: [-0.05, 0.02] da [-5 %, 0,95 × 1,02 - 1].
- `test_drawdown_series_non_positive_and_same_index`: valores ≤ 0 y el mismo índice que la entrada.
- `test_compare_with_quantstats_first_day_loss`: el mismo caso límite contra quantstats.

Pseudocódigo.

1. equity = producto acumulado de $(1 + r)$, partiendo de un capital de 1.
2. máximo previo = máximo acumulado de la equity, nunca por debajo de 1.
3. Devuelve equity / máximo previo - 1.

Pistas.

Pista 1

Compara cada día con el mejor punto alcanzado hasta entonces, incluido el punto de partida.

Pista 2

`cumprod()`, `cummax()` y, para imponer un suelo, `clip(lower=...)`. Puedes reutilizar tu `equity_curve` del TODO 2.6.

Pista 3

El suelo solo importa cuando la serie empieza perdiendo: sin él, el primer día es su propio pico y una pérdida inicial desaparece.

TODO 4.6 · max_drawdown

Qué hace. Profundidad del peor drawdown y sus fechas de pico, valle y recuperación, en un `DrawdownInfo`.

Firma.

```
def max_drawdown(returns: pd.Series) -> DrawdownInfo:
```

Convenciones que comprueban los tests.

- `test_max_drawdown_depth_and_dates`: con [0.10, -0.10, -0.10, 0.05, 0.30, -0.05], profundidad 0,891/1,1 - 1 = -19 %, pico en `d[0]`, valle en `d[2]` y recuperación en `d[4]` (`d` es el índice de fechas).
- `test_max_drawdown_from_initial_capital_has_no_peak_date`: si la caída empieza en el capital inicial, `peak` es `None`.
- `test_max_drawdown_not_recovered`: sin recuperación, `recovery` es `None`.
- `test_max_drawdown_without_losses`: sin caídas, `depth == 0` y las tres fechas `None`.
- La profundidad se compara con la de quantstats.

Pseudocódigo.

1. `dd` = serie de drawdown.
2. `profundidad` = mínimo de `dd`. Si es 0: profundidad 0.0 y las tres fechas None.
3. `valle` = primera fecha en que `dd` alcanza el mínimo.
4. `pico` = última fecha `<= valle` con `dd == 0` (None si no hay ninguna).
5. `recuperación` = primera fecha `> valle` con `dd == 0` (None si no hay ninguna).
6. Devuelve `DrawdownInfo(profundidad, pico, valle, recuperación)`.

Pistas.

Pista 1

El pico no es el máximo global de la equity, sino el último máximo antes del valle.

Pista 2

`min()`, `idxmin()` (devuelve la primera etiqueta del mínimo), `.loc[:fecha]` y `.loc[fecha:]` (ambos incluyen la fecha), filtros booleanos e `.index[0]` / `.index[-1]`.

Pista 3

`.loc[valle:]` incluye el propio valle, pero su drawdown es negativo y no molesta. Comprueba si la selección está vacía antes de pedir `.index[0]` o `.index[-1]`: vacía significa `None`, no un error.

TODO 4.7 · calmar_ratio

Qué hace. CAGR dividido entre el valor absoluto del max drawdown.

Firma.

```
def calmar_ratio(returns: pd.Series, periods: int = config.TRADING_DAYS) -> float:
```

Convenciones que comprueban los tests.

- `test_calmar_ratio_known_value`: CAGR de la serie de seis días entre 1 – 0,891/1,1 = 0,19 (resultado positivo).
- `test_calmar_ratio_without_drawdown_is_nan`: sin drawdown, NaN.

Pseudocódigo.

1. `c` = CAGR con ``periods``.
2. `d` = profundidad del max drawdown.
3. Si `d` es 0, devuelve NaN.
4. Devuelve `c / |d|`.

Pistas.

Pista 1

Reutiliza; no vuelvas a calcular nada.

Pista 2

Llama a `cagr(returns, periods)` y a `max_drawdown(returns).depth`.

Pista 3

La profundidad es negativa: sin el valor absoluto, el signo se invierte. Si el test sale *skipped* con «TODO 4.2» o «TODO 4.6», es que la función que reutilizas sigue pendiente.

TODO 4.8 · hit_ratio

Qué hace. Proporción de días ganadores entre los días con resultado distinto de cero.

Firma.

```
def hit_ratio(returns: pd.Series) -> float:
```

Convenciones que comprueban los tests.

- `test_hit_ratio_excludes_flat_days`: [0.01, 0, -0.01, 0.02, 0] da 2/3.
- `test_hit_ratio_all_flat_is_nan`: si todos los días son 0, NaN.

Pseudocódigo.

1. ganadores = número de días con $r > 0$.
2. con_resultado = número de días con $r \neq 0$.
3. Si con_resultado es 0, devuelve NaN.
4. Devuelve ganadores / con_resultado.

Pistas.

Pista 1

El denominador son los días con resultado, no todos los días.

Pista 2

La suma de una Series booleana cuenta los True.

Pista 3

«Plano» significa exactamente 0: un día fuera del mercado da 0.0 exacto porque la posición es 0, así que no hace falta tolerancia.

TODO 4.9 · annual_turnover

Qué hace. Turnover total dividido entre los años de la muestra ($n / \text{periods}$).

Firma. Lleva `@_metric(source="turnover")`: si le pasas un `BacktestResult`, usa su `.turnover`.

```
def annual_turnover(turnover: pd.Series, periods: int = config.TRADING_DAYS) -> float:
```

Convenciones que comprueban los tests.

- `test_annual_turnover_known_value`: 504 días con un turnover total de 10 dan 5.
- `summary_table` la usa para la columna «Turnover anual».

Pseudocódigo.

1. total = suma del turnover diario.
2. años = $n / \text{periods}$.
3. Devuelve total / años.

Pistas.

Pista 1

Los mismos «años» que en el CAGR.

Pista 2

`sum()` y `len()`.

Pista 3

El turnover de la cartera ya viene dividido entre N (TODO 2.5): no lo vuelvas a dividir ni lo multipliques por el número de activos.

4. Errores típicos

Desviación poblacional

Usar `np.std` sin `ddof=1`. Síntoma: volatilidad y Sharpe algo distintos (27,1 % frente a 29,7 % con la serie R). Lo detectan `test_annualized_volatility_uses_sample_std`, `test_sharpe_ratio_known_value` y la comparación con `quantstats`.

CAGR con el exponente al revés o con fechas

Elevar a `n/periods` en lugar de a `periods/n`, o contar años con días naturales. Síntoma: en el caso «medio año», $1,05^{0,5} - 1 \approx 2,47\%$ en vez de 10,25 %; con fechas, 10,36 % en vez de 10 %. Lo detecta `test_cagr_known_values`.

rf / 252

Convertir el tipo anual a diario dividiendo. Síntoma: un Sharpe ligeramente menor (4,0743 frente a 4,0783 con `rf = 5 %`). Lo detectan `test_sharpe_ratio_geometric_risk_free` y `test_compare_with_quantstats_matches[0.03]`.

Sharpe infinito o con error en una serie constante

Dividir sin comprobar σ . Síntoma: `inf`, `NaN` con aviso o `ZeroDivisionError` si conviertes a float de Python. Lo detecta `test_sharpe_ratio_constant_series_is_nan`.

Sortino con otro denominador

Dividir entre el número de días negativos (5,02 con la serie R) o usar la desviación típica de los rendimientos negativos (11,22) en lugar de dividir entre `n` (8,69). Lo detectan `test_sortino_ratio_known_value` y `quantstats`.

Sortino sin restar rf en la parte bajista

Calcular el exceso para la media pero tomar `min(r, 0)` sobre el rendimiento bruto. Síntoma: solo falla con `rf > 0`. Lo detectan `test_sortino_ratio_with_risk_free` y `test_compare_with_quantstats_matches[0.03]`.

Olvidar el capital inicial

Calcular el máximo acumulado solo sobre la equity. Síntoma: una pérdida el primer día da `drawdown 0`. Lo detectan `test_drawdown_series_initial_capital_is_a_peak`, `test_max_drawdown_from_initial_capital_has_no_peak_date` y `test_compare_with_quantstats_first_day_loss`.

Drawdown con el signo cambiado

Calcular `M/E - 1` o `1 - E/M`. Síntoma: valores positivos. Lo detecta `test_drawdown_series_non_positive_and_same_index`.

Pico = máximo global

Tomar como pico la fecha del máximo de toda la equity. En la serie de `test_max_drawdown_depth_and_dates` ese máximo está en `d[4]`, después del valle (`d[2]`). Lo detecta ese mismo test.

Calmar negativo

Dividir entre la profundidad sin valor absoluto. Lo detecta `test_calmar_ratio_known_value`.

Contar los días planos

Dividir entre todos los días. Síntoma: $2/5 = 0,4$ en lugar de $2/3$. Lo detecta `test_hit_ratio_excludes_flat_days`.

252 escrito a mano

Si usas 252 en vez de `periods`, los tests pasan (todos usan el valor por defecto), pero `summary_table(..., periods=...)` y cualquier análisis con datos semanales o mensuales darán resultados falsos sin avisar.

5. Preguntas y ejercicios

1. ¿Por qué la media diaria se anualiza multiplicando por 252 y la volatilidad por $\sqrt{252}$? ¿Qué factor queda para el Sharpe?
2. Una estrategia tiene autocorrelación diaria de primer orden $\rho = 0,2$. ¿Su volatilidad anual real es mayor o menor que $\sigma\sqrt{252}$? ¿Y su Sharpe real?
3. Dos estrategias tienen la misma media aritmética anual, un 10 %, con volatilidades del 10 % y del 30 %. ¿Qué CAGR aproximado tiene cada una?
4. ¿Por qué el Sortino con n total sale mayor que dividiendo solo entre los días negativos? ¿Cuál de los dos es más estable cuando hay muy pocas pérdidas?
5. En la tabla de drawdown de la sección 2, ¿qué devolvería `max_drawdown` si la serie acabara en el día 4?
6. ¿Por qué el Calmar de un histórico largo tiende a ser menor que el de uno corto?
7. ¿Cómo puede ganar dinero una estrategia con un hit ratio del 40 %? ¿Y perderlo con uno del 70 %?
8. Con un turnover anual de 20 y costes de $5 + 5$ bps, ¿cuánto rendimiento anual se lleva el broker?
9. Ejercicio (notebook 02, sección 3): completa la celda `cost_check` comparando turnover anual $\times 10 / 10\,000$ con la diferencia de CAGR entre la tabla sin costes y la tabla con costes. ¿Por qué no coinciden exactamente?
10. Ejercicio (notebook 02): para cada estrategia, calcula el error estándar aproximado del Sharpe ($1/\sqrt{\text{años}}$) y decide qué diferencias de la tabla comparativa se distinguen del ruido. Ejecuta `compare_with_quantstats` sobre cada `BacktestResult`.

6. Checkpoint

```
python -m pytest tests/test_metrics.py
python -m pytest tests/test_metrics.py -rs
python -m pytest -k annualized_volatility
python -m pytest -k sharpe
python -m pytest -k sortino
python -m pytest -k drawdown_series
python -m pytest -k max_drawdown
python -m pytest tests/test_quantstats_check.py
```

Igual que en el capítulo 03, los tests que llegan a un TODO sin hacer salen como *skipped* con el motivo «TODO pendiente: TODO 4.x · ...», y al final se listan los TODO pendientes; `-rs` muestra el motivo de cada uno y `--todo-fail` los convierte en fallos. El motivo dice qué TODO bloquea el test, que no siempre es el del nombre: un test de Calmar puede esperar al 4.2 o al 4.6.

`tests/test_metrics.py` tiene 24 tests y al principio salen los 24 como skipped. Por TODO: 4.1, 2 tests; 4.2, 3; 4.3, 4; 4.4, 3; 4.5, 3; 4.6, 4; 4.7, 2; 4.8, 2; 4.9, 1. Al terminar, 24 passed. `tests/test_quantstats_check.py` tiene 4 tests y se salta entero si quantstats no está instalado (`python -m pip install quantstats`). Necesitan los TODO 4.1 y 4.3 a 4.6; el último, `test_compare_with_quantstats_on_backtest`, además el motor (1.1 y 2.1 a 2.6) y el cruce de medias (3.1). Cuando todo esté hecho, 4 passed. Si alguno falla, el mensaje imprime la tabla de `compare_with_quantstats`: la fila con la diferencia grande te dice qué convención revisar.